

LAB TASK#7

NAME: MASHAAL ISHTIAQ

ROLL NO: 22P-9326

SECTION: BCS-4C

QUESTION#1

CODE:

```
#include <iostream>
using namespace std;

class Node
{
public:
int data;
Node *next;
Node(int d)
{
data = d;
next = NULL;
}
};

class Queue
{
private:
int size;
Node *front;
Node *rear;

public:
Queue()
{
size = 0;
front = NULL;
rear = NULL;
}

bool isEmpty()
{
return size == 0;
}

bool isFull()
{
// As the queue is implemented using a linked list, it can never be full.
return false;
}

void Display()
{
if (isEmpty())
```

```
{
cout << "Queue is empty." << endl;
return;
}
Node *temp = front;
cout << "Orders in Queue:" << endl;
while (temp != NULL)
{
cout << temp->data << " ";
temp = temp->next;
}
cout << endl;
}

void Enqueue(int d)
{
Node *newNode = new Node(d);
if (!isFull())
{
if (isEmpty())
{
front = rear = newNode;
}
else
{
rear->next = newNode;
rear = newNode;
}
size++;
}
else
{
cout << "Queue is full. Cannot add more orders." << endl;
}
}

void Dequeue()
{
if (isEmpty())
{
cout << "Queue is empty. No orders to process." << endl;
return;
}
Node *temp = front;
front = front->next;
delete temp;
size--;
}
};

int main()
{
```

```
Queue q;
int choice;

cout << "____KASHMIRI CAFE(FAST PWR CAMPUS)____" << endl;
do
{
cout << "Menu Options:" << endl;
cout << "1. Add New Order" << endl;
cout << "2. Process Order" << endl;
cout << "3. Display Current Orders" << endl;
cout << "4. Exit" << endl;
cout << "Enter your choice: " << endl;
cin >> choice;

switch (choice)
{
case 1:
{
cout << " Menu For Nashta " << endl;
cout << "1. chai (Rs.80)" << endl;
cout << "2. Paratha (Rs.100)" << endl;
cout << "3. channay (Rs.120)" << endl;
cout << "4. roll (Rs.30)" << endl;
cout << "5. samosa (Rs.25)" << endl;
int order;
cout << "Select your order: ";
cin >> order;
q.Enqueue(order); // order number will be added to the queue
break;
}
case 2:
{
if (!q.isEmpty())
{
cout << "Processing order.. " << endl;
q.Dequeue(); // removing the order from the queue
q.Display();
cout << endl;
}
else
{
cout << "No orders in queue to process." << endl;
}
break;
}
case 3:
q.Display();
break;
case 4:
cout << "Exiting program..." << endl;
break;
default:
```

```
cout << "Invalid choice. Please enter a valid option." << endl;
}
} while (choice != 4);

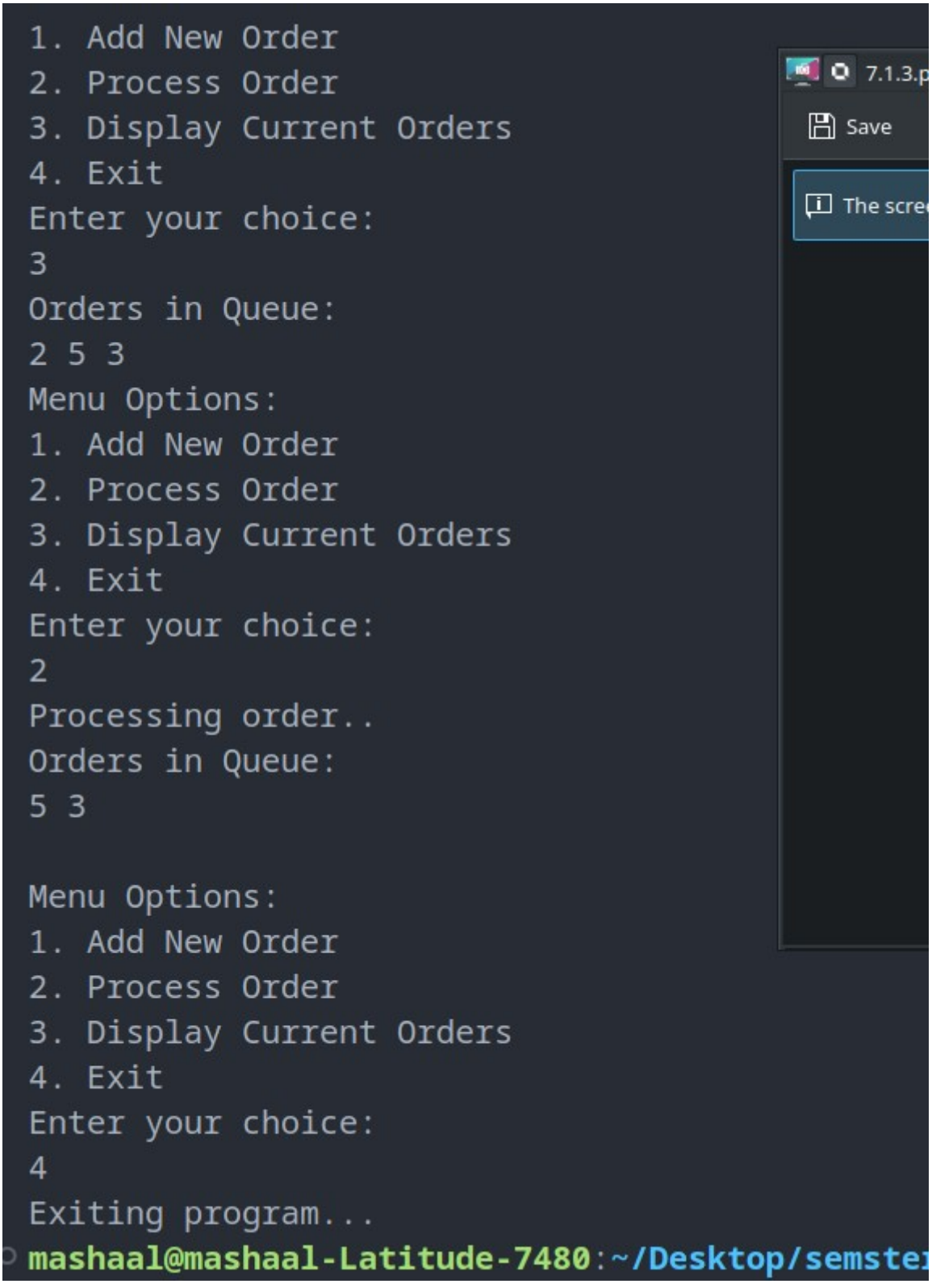
return 0;
}
```

```
ile.cpp -o tempCodeRunnerFile && "/home/mashaa
____KASHMIRI CAFE(FAST PWR CAMPUS)____
Menu Options:
1. Add New Order
2. Process Order
3. Display Current Orders
4. Exit
Enter your choice:
1
 Menu For Nashta
1. chai (Rs.80)
2. Paratha (Rs.100)
3. channay (Rs.120)
4. roll (Rs.30)
5. samosa (Rs.25)
Select your order: 1
Menu Options:
1. Add New Order
2. Process Order
3. Display Current Orders
4. Exit
Enter your choice:
1
 Menu For Nashta
1. chai (Rs.80)
2. Paratha (Rs.100)
3. channay (Rs.120)
```

```
5. samosa (Rs.25)
Select your order: 2
Menu Options:
1. Add New Order
2. Process Order
3. Display Current Orders
4. Exit
Enter your choice:
1
 Menu For Nashta
1. chai (Rs.80)
2. Paratha (Rs.100)
3. channay (Rs.120)
4. roll (Rs.30)
5. samosa (Rs.25)
Select your order: 5
Menu Options:
1. Add New Order
2. Process Order
3. Display Current Orders
4. Exit
Enter your choice:
3
Orders in Queue:
1 2 5
Menu Options:
1. Add New Order
2. Process Order
```

```
4. Exit
Enter your choice:
2
Processing order..
Orders in Queue:
2 5

Menu Options:
1. Add New Order
2. Process Order
3. Display Current Orders
4. Exit
Enter your choice:
1
 Menu For Nashta
1. chai (Rs.80)
2. Paratha (Rs.100)
3. channay (Rs.120)
4. roll (Rs.30)
5. samosa (Rs.25)
Select your order: 3
Menu Options:
1. Add New Order
2. Process Order
3. Display Current Orders
4. Exit
Enter your choice:
3
```


1. Add New Order
2. Process Order
3. Display Current Orders
4. Exit
Enter your choice:
3
Orders in Queue:
2 5 3
Menu Options:
1. Add New Order
2. Process Order
3. Display Current Orders
4. Exit
Enter your choice:
2
Processing order..
Orders in Queue:
5 3
Menu Options:
1. Add New Order
2. Process Order
3. Display Current Orders
4. Exit
Enter your choice:
4
Exiting program...
mashaal@mashaal-Latitude-7480:~/Desktop/semste
7.1.3.p
Save
The scre

QUESTION#2

CODE:

```
#include <iostream>
using namespace std;

class Node
{
public:
int data;
Node *next;
Node(int d)
{
data = d;
next = NULL;
}
};

class Queue
{
private:
Node *front;
Node *rear;

public:
Queue()
{
front = NULL;
rear = NULL;
}

~Queue()
{
Node *temp = front;
while (temp != NULL)
{
front = front->next;
delete temp;
temp = front;
}
}

bool isEmpty()
{
```

```
return front == NULL;
}

void Enqueue(int d)
{
Node *newNode = new Node(d);
if (isEmpty())
{
front = rear = newNode;
}
else
{
rear->next = newNode;
rear = newNode;
}
}


void Display()
{
if (isEmpty())
{
cout << "Queue is empty." << endl;
return;
}

Node *temp = front;
while (temp != NULL)
{

if (temp != front)
{
cout << "->";
}
cout << temp->data;
temp = temp->next;
}
cout << endl;
}

// Function to duplicate nodes in the queue based on their data
void DuplicateQueue()
{
Node *curr = front;
while (curr != NULL)
{
int data = curr->data;
for (int i = 1; i < data; i++)
{
Node *newNode = new Node(data);
newNode->next = curr->next;
curr->next = newNode;
```

```
curr = curr->next;

if (curr == rear)
{
rear = newNode;
}
}
curr = curr->next;
}
}
};

int main()
{
Queue q;
char ch = 'Y';

cout << "Enter numbers into the queue (-1 to stop):" << endl;
int num;
while (true)
{
cin >> num;
if (num == -1)
break;
q.Enqueue(num);
}
// q.Enqueue(2);
// q.Enqueue(3);
// q.Enqueue(2);

cout << "Original Queue: ";
q.Display();

// Duplicate the queue
q.DuplicateQueue();

cout << "Duplicated Queue: ";
q.Display();

return 0;
}
```

```
alIshtiaq_task#7.2 && "/home/mashaal/Desktop/se
Enter numbers into the queue (-1 to stop):
2
3
2
-1
Original Queue: 2->3->2
Duplicated Queue: 2->2->3->3->3->2->2
mashaal@mashaal-Latitude-7480:~/Desktop/semster
```

```
Ishtiaq_task#7.2.cpp -o 22p9326_mashaalIshtiaq_
Enter numbers into the queue (-1 to stop):
2
3
3
-1
Original Queue: 2->3->3
Duplicated Queue: 2->2->3->3->3->3->3->3
```